

REA Software Developer Assessment Preparation

Diagnostic Test, Corrections & Python Practice

Assessment: Rural Electrification Authority (REA) — Software Developer Professional Assessment Test

Scheduled: Wednesday, 26 August 2026 at 11:00 hours

Preparation focus: Programming, SQL, OOP, APIs, SDLC, testing, security and systems analysis

1. Preparation Strategy

The preparation approach is based on Learn → Code → Verify: study the concept, solve a problem without looking up the answer, review the result, and then increase the difficulty. The objective is to improve both conceptual understanding and technical precision.

Priority Areas

Area	Priority	Current assessment
Programming & problem solving	Very High	Needs continued practice
SQL & databases	Very High	Good foundation; improve precision
OOP	Very High	Good
SDLC & requirements	Very High	Strong
Web & REST APIs	High	Strong
Testing	High	Needs revision
Systems analysis/design	High	Needs deeper practice
Security	Medium	Good foundation

2. Diagnostic Test — Round 1

Diagnostic score: 14/20 (70%).

The diagnostic showed a good practical foundation. Strong areas included APIs, authentication/authorization, SDLC, requirements, classes/objects and basic SQL. Main gaps were technical precision, algorithmic reasoning, testing and some SQL details.

Question-by-question corrections

1. Compiler vs interpreter

A compiler translates source code into a lower-level form such as machine code or an intermediate representation before execution. An interpreter executes/evaluates source code at runtime. Assembly language is not the same as machine code.

2. Python list comprehension

Correct: [4, 8].

3. OOP principles

The four principles are Encapsulation, Inheritance, Polymorphism and Abstraction. Encapsulation was missing from the original answer.

4. Class vs object

Correct. A class is a blueprint; an object is an instance of that class.

5. Prime number

The original logic was incorrect because `number % number == 0` is true for every non-zero number. A divisor search is required.

6. Primary vs foreign key

Correct concept. A primary key uniquely identifies a record; a foreign key references a key in another table.

7. SQL salary query

Correct: `SELECT * FROM employees WHERE salary > 10000`; Avoid `SELECT * AS ...` for aliasing the entire result.

8. INNER JOIN vs LEFT JOIN

INNER JOIN returns matching rows from both tables. LEFT JOIN returns all rows from the left table plus matching right-side rows; unmatched right-side values are NULL.

9. Normalization

Normalization organizes data into related tables to reduce redundancy, prevent anomalies and improve data integrity.

10. WHERE vs HAVING

WHERE filters rows before grouping. HAVING filters groups after GROUP BY. Memory aid: WHERE → rows; HAVING → groups.

11. API

An API is a defined interface that allows software systems to communicate. It is not limited to communication with a database.

12. GET vs POST

Correct: GET retrieves a resource; POST submits/creates a resource.

13. HTTP codes

200 OK; 201 Created; 400 Bad Request; 401 authentication issue; 403 forbidden; 404 not found; 500 server error.

14. Authentication vs authorization

Correct: authentication answers 'Who are you?' while authorization answers 'What are you allowed to access/do?'

15. PUT vs PATCH

Correct: PUT generally replaces/updates the complete resource; PATCH partially updates a resource.

16. SDLC

Good answer: requirements → analysis → design → development → testing → deployment → maintenance.

17. Functional vs non-functional requirements

Correct. Functional requirements describe what the system does; non-functional requirements describe qualities or constraints such as response time, security or availability.

18. Unit vs integration testing

Not answered. Unit testing tests an individual component/function; integration testing verifies that multiple components work together.

19. REA project tracking system

Good functional requirements included login, project creation, progress tracking, reporting and storing project/location information.

20. Slow application

Good starting points: database queries/indexes, server resources, network latency, API response time, large resources, frontend rendering, concurrency, logs and caching.

3. Python Practice — Challenge 1: Prime Numbers

The first attempt incorrectly treated divisibility by 2 as proof of primality. The corrected solution checks possible divisors.

```
def is_prime(number):
    if number < 2:
        return False

    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            return False

    return True
```

Key lesson: A prime number is greater than 1 and has exactly two factors: 1 and itself. Checking up to the square root is an efficient approach.

4. Python Practice — Challenge 2: Palindrome

The palindrome solution was logically correct. The issue was returning a string instead of the Boolean value False.

```
def is_palindrome(text: str):
    reversed_text = text[::-1]

    if text == reversed_text:
        return True

    return False

# Cleaner version:
def is_palindrome(text: str):
    return text == text[::-1]
```

Key lesson: True and False are Boolean values; "True" and "False" are strings.

5. Python Practice — Challenge 3: Frequency Counting

The solution using list.count() correctly produced {1: 1, 2: 2, 3: 3, 4: 1}. The preferred assessment solution uses the dictionary itself as the counter.

```
def count_frequency(numbers):
    frequency = {}

    for num in numbers:
        if num in frequency:
            frequency[num] += 1
        else:
            frequency[num] = 1

    return frequency
```

Key lesson: This demonstrates loops, dictionaries, conditionals and efficient counting.

6. Core Technical Revision Notes

OOP: Encapsulation, Inheritance, Polymorphism, Abstraction.

SQL: SELECT, WHERE, GROUP BY, HAVING, JOINS, aggregate functions, primary/foreign keys and normalization.

REST: GET retrieves, POST creates/submits, PUT replaces/updates the complete resource, PATCH partially updates, DELETE removes.

HTTP: 200 OK, 201 Created, 400 Bad Request, 401 Authentication issue, 403 Forbidden, 404 Not Found, 500 Server Error.

SDLC: Requirements → Analysis → Design → Development → Testing → Deployment → Maintenance.

Testing: Unit tests one component; integration tests components together; system testing tests the complete system; acceptance testing verifies business/user requirements; regression testing ensures changes have not broken existing functionality.

Security: Authentication, authorization, hashing, encryption, HTTPS, SQL injection, XSS, CSRF, input validation and access control.

Performance: Investigate database queries/indexes, server CPU/RAM, network latency, API response time, frontend rendering, large resources, concurrent users, logs and caching.

7. REA Scenario Practice

Scenario: REA wants a web-based system for monitoring rural electrification projects across Zambia. The system should allow REA officers to register projects, record project locations, track progress, upload documents, generate reports and allow management to monitor project status.

A strong development approach:

1. Requirements gathering
2. System analysis
3. Database design
4. System architecture
5. UI/UX design
6. Backend/API development
7. Frontend development
8. Authentication & authorization
9. Testing
10. Deployment
11. Documentation
12. Maintenance

Possible functional requirements: user authentication; project creation; project progress updates; project location capture; document uploads; project reporting; project status monitoring.

8. Final Preparation Checklist

- Revise Python functions, loops, lists, dictionaries and strings.
- Practice prime numbers, palindrome, frequency counting and second-largest-number problems.
- Revise SQL WHERE vs HAVING, GROUP BY, JOINS and aggregate functions.
- Revise the four OOP principles.
- Revise HTTP methods and status codes.
- Revise authentication vs authorization.
- Revise SDLC and requirements engineering.
- Revise unit, integration, system, acceptance and regression testing.
- Practice one REA system-design scenario.
- Complete a timed mock assessment before the actual test.

9. Assessment-Day Reminders

Venue: NIPA Library, Dushambe Road, Lusaka

Date: Wednesday, 26 August 2026

Time: 11:00 hours

Arrival target: 10:30 hours

Bring: valid NRC/Passport/Driver's Licence, pen and notebook.

Preparation principle: Do not only memorize definitions. Explain concepts clearly, write working code, and show your reasoning when solving problems.